

Phase 27A — Planner Promotion Readiness Audit

Audit mode: No code, tests, UI, source-of-truth, persistence, or implementation files were modified.

Planner status: developer-only, advisory-only, read-only, not promoted.

1. Promotion readiness score

38 / 100

Verdict

The planner is not ready to replace the legacy training engine yet.

It has made strong progress as a developer-only advisory prescription model, especially around RecoveryDay preservation, core support extraction, deload metadata, duration fidelity, primary session resolution, and preview/train-screen developer models.

However, it is still not safe as the app's canonical runtime engine because the active app still depends on multiple legacy paths for Home display, Train execution, run/lift logging, readiness/recommendation/progression feedback loops, persistence, and source-of-truth state shape.

The planner currently consumes legacy engine outputs more than it owns them.

2. Home integration readiness

Current source of truth

- `src/app/page.tsx` : `todayWorkout` , `adjustedTodayWorkout` , `todayRunDisplay` , `runningRecommendation` , `dailyPrescription` , `homeCommandCenter`
- `src/lib/home-command-center.ts` : `buildHomeCommandCenter` , `getTodayRunForDate` , `getScheduledRunForTraining` , calls to `readiness/progression/goal/training` evaluation
- `src/lib/training-engine.ts` : `evaluateTraining`
- `src/lib/coach-engine.ts` : `generateDailyPrescription` , `adjustWorkoutForReadiness` , `generateRunningRecommendation`

Planner equivalent

- `src/lib/training-planner.ts` : `buildDailyTrainingSession` , `DailyTrainingSession`
- `src/lib/training-planner-adapter.ts` : `buildPlannerSessionFromAppState`
- `src/lib/planner-shadow-observability.ts` : compares planner to legacy
- `src/lib/planner-train-preview.ts` : developer-only preview model
- `src/lib/planner-train-screen-v1.ts` : developer-only train-screen model

Missing capabilities

- Home does not consume `DailyTrainingSession` as canonical daily session.
- Home still separately computes today workout, today run, daily prescription, running recommendation, and training engine output.
- Planner has no promoted Home adapter equivalent to `HomeCommandCenterModel` .
- Mission Control still consumes `homeCommandCenter.trainingEngineResult` , not planner output.
- Planner is only built when `plannerDebugEnabled` is true.

Risks

- Home can show a session that diverges from planner.
- Planner may classify a day correctly while Home continues to display legacy adjusted workout/run labels.
- Duration/run fidelity improvements in planner do not automatically improve Home recommendation behavior.
- If planner is promoted without a Home adapter, dashboard and Train could disagree.

Blockers

- Critical: Home runtime is still legacy-source-driven, not planner-source-driven.
- Critical: Planner has no production DailyTrainingSession -> HomeCommandCenterModel adapter.
- Major: buildHomeCommandCenter independently runs training/progression/goal/readiness logic.
- Major: Home still uses getTodayRunForDate and getScheduledRunForTraining , not planner run logging/prescription metadata.

Exact files involved

- src/app/page.tsx
- src/lib/home-command-center.ts
- src/lib/home-daily-dashboard.ts
- src/lib/training-engine.ts
- src/lib/coach-engine.ts
- src/lib/training-planner.ts
- src/lib/training-planner-adapter.ts
- src/lib/planner-shadow-observability.ts

3. Train integration readiness

Current source of truth

- src/app/page.tsx : TrainScreen , TrainingPlan , ActiveWorkout , TrainRunLogger
- src/lib/training-engine.ts : TrainingEngineResult
- src/lib/run-logger.ts : saveTrainRunLog
- src/lib/coach-engine.ts : starting weight, next-set recommendation, next workout step, post-workout analysis

Train execution still starts from the legacy Workout object: displayedWorkout.id , displayedWorkout.title , displayedWorkout.exercises .

Planner equivalent

- src/lib/planner-train-preview.ts
- src/lib/planner-train-screen-v1.ts
- src/app/page.tsx : PlannerTrainPreviewDeveloperPanel , PlannerTrainScreenV1DeveloperPanel

These are developer-only, read-only, advisory-only, visible behind plannerDebugEnabled , and not executable.

Missing capabilities

- No production Train screen backed by DailyTrainingSession .
- No planner execution adapter that maps workout/run logging targets, blocks, support, and deload metadata into runtime logging.
- ActiveWorkout still expects legacy Workout shape.
- Planner support blocks are visible but not executable/loggable.

- Hybrid behavior is not promotion-safe: planner train-screen lift logging appears only for LiftDay , which could hide lift logging for HybridDay .

Risks

- User could execute a legacy workout while planner says a different session should happen.
- Core/support/conditioning visibility in planner does not guarantee execution or logging.
- Planner logging targets exist as metadata, but Train does not use them for production behavior.
- Hybrid or mixed-modality days could lose logging affordances.

Blockers

- Critical: Train execution is not planner-backed.
- Critical: Planner train screen is read-only preview, not runtime execution.
- Critical: Existing active workout logging requires legacy Workout.exercises , not planner prescription blocks.
- Major: Planner has no stable execution adapter for lift/run/support/conditioning/mobility blocks.
- Major: Hybrid/mixed-session logging rules are not fully promotion-safe.
- Major: Planner support work is not connected to any execution state.

Exact files involved

- src/app/page.tsx
- src/lib/training-engine.ts
- src/lib/training-planner.ts
- src/lib/planner-train-preview.ts
- src/lib/planner-train-screen-v1.ts
- src/lib/run-logger.ts
- src/lib/workout-logger.ts
- src/lib/coach-engine.ts
- src/lib/types.ts

4. Log integration readiness

Current source of truth

- src/app/page.tsx : Log screen, check-in, nutrition, body metrics, photos, Train run logger, active workout
- src/lib/run-logger.ts
- src/lib/workout-logger.ts
- src/lib/daily-checkin.ts
- src/lib/log-tab-ui.ts
- src/lib/storage.ts
- src/lib/types.ts

Persisted log data lives in AppState : check-ins, run logs, workout sessions, set logs, summaries, post-workout recommendations, adjustments, nutrition, body, and photo logs.

Planner equivalent

- DailyWorkoutPrescription.loggingTarget

- DailyRunPrescription.loggingTarget
- DailyTrainingBlock.executionTarget
- Defined in src/lib/training-planner.ts

Missing capabilities

- No production mapper from planner logging targets to RunLog , WorkoutSession , SetLog , or WorkoutSummary .
- No planner-backed completion reconciliation.
- Planner completion status is read-only and computed from legacy logs.
- No log UI that says “log this planner block.”
- No migration for existing logs to planner session references.
- No source-preserving audit trail connecting logged work back to a planner session ID.

Risks

- Promotion could break daily completion detection.
- Existing run/lift logs may no longer match planner prescription IDs.
- Duplicate or missing logs possible if planner and legacy paths coexist.
- Support work could be visible in planner but untracked in Log.

Blockers

- Critical: Planner logging targets are not production logging sources.
- Critical: AppState has no persisted DailyTrainingSession or planner session reference for logs.
- Major: Existing logging flows still depend on legacy workout/run structures.
- Major: No migration strategy for old logs.
- Major: No planner-backed completion status in persisted state.
- Major: No rollback/backup plan specific to planner promotion.

5. Recommendation integration readiness

Current source of truth

- src/lib/coach-engine.ts : daily prescription, running recommendation, next-set recommendation, post-workout analysis, readiness adjustment
- src/app/page.tsx : dailyPrescription , runningRecommendation , automatic adjustments
- src/lib/running-engine.ts
- src/lib/workout-engine.ts

Planner equivalent

- BuildDailyTrainingSessionInput.runningRecommendation
- DailyTrainingSession.modifications
- DailyTrainingSession.warnings
- DailyTrainingSession.todayGoals
- DailyTrainingSession.run.runningRecommendationAction

The planner carries recommendation inputs and modifications, but does not own recommendation generation.

Missing capabilities

- Planner does not replace generateDailyPrescription .
- Planner does not replace generateRunningRecommendation .
- Planner does not own next-set or post-workout recommendations.
- Planner consumes runningRecommendation ; it does not generate it.
- Recommendation side effects still write adjustments from legacy dailyPrescription .

Risks

- Circular dependency if planner replaces training engine before recommendation sources are reconciled.
- Planner could recommend one session while legacy coach recommendations tell user another thing.
- Running recommendation currently does not run for zero-distance duration-based runs.
- Automatic adjustment log entries could record legacy decisions that conflict with planner decisions.

Blockers

- Critical: Recommendation engine ownership is unresolved.
- Critical: Planner is not the source of daily prescription recommendations.
- Critical: Recommendation side effects still come from legacy dailyPrescription .
- Major: Running recommendations are distance-centric in the active app path.
- Major: Next-set and post-workout recommendations are not planner-aware.
- Major: No canonical conflict-resolution policy between planner and recommendation engines.

6. Readiness integration readiness

Current source of truth

- src/app/page.tsx : calculateReadiness(latestCheckIn, baseline) , adjusted workout, Home/Train readiness
- src/lib/coach-engine.ts : calculateReadiness
- src/lib/readiness-engine.ts : evaluateReadiness , readinessInputFromDailyCheckIn
- src/lib/daily-checkin.ts
- src/lib/home-command-center.ts

Planner equivalent

- BuildDailyTrainingSessionInput.readinessResult
- DailyTrainingSession.readinessStatus
- DailyTrainingSession.status
- DailyTrainingSession.modifications
- DailyTrainingSession.warnings

Missing capabilities

- Planner does not own readiness calculation.
- Planner adapter fabricates fallback readiness if not provided.
- No production policy for stale/missing readiness when planner is canonical.
- Active UI still applies adjustWorkoutForReadiness separately before planner.

Risks

- Double-modification risk if planner is promoted without removing legacy readiness adjustment.
- Fallback readiness could be unsafe if accidentally used in production.
- Readiness status can be computed in multiple places.

Blockers

- Critical: Readiness ownership and call sequence are not promotion-safe.
- Critical: Planner depends on readiness input; it does not define readiness truth.
- Major: Duplicate readiness pathways need consolidation.
- Major: Planner fallback readiness should remain shadow-only or be replaced by a production-safe missing-data policy.

7. Progression integration readiness

Current source of truth

- `src/lib/progression-engine.ts` : `evaluateProgression`
- `src/lib/home-command-center.ts` : weekly progression input
- `src/lib/weekly-review.ts`
- `src/lib/running-engine.ts`
- `src/lib/workout-engine.ts`
- `src/app/page.tsx` : passes progression result into planner shadow only

Planner equivalent

- `BuildDailyTrainingSessionInput.progressionResult`
- `DailyTrainingSession.modifications`
- `DailyTrainingSession.warnings`
- Recovery override when weekly decision is Recovery Focus

Missing capabilities

- Planner does not own weekly progression calculation.
- Planner cannot yet drive week-to-week plan modifications.
- Planner does not persist progression decisions.
- Planner does not replace `evaluateProgression` .
- No promotion-safe contract defining whether progression modifies daily prescription, next week plan, source training plan, recommendation copy, or logging expectations.

Risks

- Progression decisions could be applied twice or ignored.
- Deload/recovery policy could diverge between planner metadata and progression engine decision.
- Long-run/running progression can still be decided outside planner while planner owns run prescription.

Blockers

- Critical: Progression integration is unresolved as a source-of-truth relationship.
- Critical: Planner cannot safely replace training engine until progression decision ownership is explicitly staged.
- Major: No planner-backed weekly planning/promotion path.
- Major: No persisted progression audit attached to planner session.

8. Persistence integration readiness

Current source of truth

- `src/lib/types.ts` : AppState
- `src/lib/storage.ts` : migration, load recovery, save state, localStorage snapshots
- `src/lib/supabase-persistence.ts` : Supabase sync
- `src/lib/backup-restore.ts`

Planner equivalent

Planner has no persisted canonical state. DailyTrainingSession is generated ephemerally from app state, seed workouts, readiness, progression, goal tracking, running recommendation, and logs.

Missing capabilities

- No AppState.dailyTrainingSessions .
- No persisted planner session ID.
- No persisted planner decision/audit artifact.
- No migration plan.
- No Supabase schema mapping for planner sessions.
- No backup/restore schema inclusion for planner sessions.
- No rollback strategy if planner promotion causes mismatched logs.

Risks

- Promoted planner decisions would be non-reproducible if source inputs change.
- Logs could reference legacy workout IDs while UI references planner session IDs.
- Backup/restore cannot preserve planner session decisions.
- Supabase sync cannot persist planner sessions.

Blockers

- Critical: Persistence has no planner source-of-truth model.
- Critical: Planner is generated-only and cannot be safely used as canonical runtime state.
- Major: No migration/rollback plan.
- Major: No Supabase schema contract.
- Major: No backup/restore coverage for planner promotion.

9. Critical blockers summary

- Planner is not the runtime source of truth. Home and Train still build from legacy paths.
- Train execution is still legacy-workout based. No executable production Train adapter exists.
- Logging targets are planner metadata only. Runtime logging still writes through legacy forms.
- Recommendation ownership is unresolved. Planner consumes recommendation outputs but does not replace them.
- Readiness/progression ownership is unresolved. Planner consumes these outputs but active app computes and applies them separately.
- Persistence has no planner state. No persisted planner session or planner audit artifact exists.

10. Major blockers summary

- Home model adapter missing.
- Planner Train UI is preview-only.
- Hybrid/mixed modality logging needs hardening.
- Duration-based run recommendation gap remains active-path risk.
- Source-plan ownership remains seed-based.
- Shadow/promotion audit tools are not sufficient as promotion gates.

11. Minor blockers summary

- Duplicate planner readiness construction in page.tsx .
- Regex classification still exists.
- UTC day-index assumptions should be validated before source-of-truth promotion.
- Developer-only UI language needs production removal or conversion.
- Planner source provenance is incomplete.

12. Estimated phases remaining before planner promotion

Estimated remaining work: 7–9 phases before safe promotion.

- Phase 27B — Planner Promotion Contract
- Phase 27C — Home Adapter Readiness
- Phase 27D — Train Execution Adapter
- Phase 27E — Log/Persistence Mapping
- Phase 27F — Recommendation Ownership Reconciliation
- Phase 27G — Readiness/Progression Ownership Reconciliation
- Phase 27H — Full Shadow Trial + Safety Gates
- Phase 28A — Feature-flagged Planner Runtime Pilot
- Phase 28B — Promotion / Legacy Fallback Removal Plan

I would not promote before Phase 28A at the earliest.

13. Recommended promotion roadmap

Phase 27B — Planner Promotion Contract

Audit/design only. Define canonical ownership boundaries, planner input/output contracts, remaining engine ownership, planner non-mutation rules, migration requirements, and rollback requirements.

Deliverable: PLANNER_PROMOTION_CONTRACT_V1.md . No code changes.

Phase 27C — Home Adapter Plan + Implementation

Create a production-safe adapter: DailyTrainingSession -> HomeCommandCenterModel-compatible training fields .

Required proofs: Home workout from planner, Home run from planner, duration-based runs display correctly, RecoveryDay displays as recovery, deload metadata appears only as metadata, and no planner promotion unless behind explicit flag.

Phase 27D — Train Execution Adapter

Create adapter: DailyTrainingSession -> executable Train model .

Must support lift blocks, run blocks, recovery blocks, mobility blocks, conditioning blocks, support/core blocks, logging visibility, logging defaults, and hybrid behavior without duplicate legacy adjustment.

Phase 27E — Log + Persistence Mapping

Design and implement safe persistence model. Decide whether to persist full session, planner session ID only, planner audit hash, source workout ID plus planner version, or logging target references. Update localStorage migration, Supabase sync contract, backup/restore, and rollback plan.

Phase 27F — Recommendation Ownership Reconciliation

Define whether planner consumes, wraps, owns final daily copy, owns adjustment entries, or triggers run/workout recommendations. Eliminate conflicts between planner, daily prescription, running recommendation, post-workout recommendations, and automatic adjustments.

Phase 27G — Readiness/Progression Ownership Reconciliation

```
AppState/logs/check-in -> readiness/progression engines -> planner -> Home/Train/Log
```

Prevent double readiness adjustment, fallback readiness in production, and planner/legacy mismatch on recovery/deload days.

Phase 27H — Full Runtime Shadow Trial

Run planner and legacy side by side across all 84 seed days, readiness states, progression modes, deload metadata, duration runs, long runs, lift days, conditioning days, core support days, hybrid/mixed days, and logged/completed states.

Promotion gate should require zero critical logging mismatches, zero source-of-truth conflicts, zero persistence regressions, and documented expected differences only.

Phase 28A — Feature-flagged Planner Runtime Pilot

Use a hard feature flag: plannerRuntimeEnabled . Preserve legacy fallback, avoid data migration without backup, compare planner-vs-legacy runtime outputs, capture mismatches, and allow rollback.

Phase 28B — Promotion Decision

Promote only if Home uses planner, Train uses planner, Log uses planner logging targets, recommendations are reconciled, readiness/progression sequence is fixed, persistence/backup/rollback are proven, and shadow trial has no critical blockers.

14. Final recommendation

Do not promote planner yet.

The planner is strong enough to continue as a developer-only advisory/shadow system, but it is not yet safe to replace the legacy training engine.

Recommended next phase: Phase 27B — Planner Promotion Contract V1.

That phase should be audit/design-only and should define the exact promotion contract before any runtime integration changes.